

Ticket Booking System

A Microservices-Based Architecture

Team Members

1. *Muhtasim Zawad - 220042131*
2. *Abdullah Al Musaddiq - 220042135*
3. *Golam Mahadi Ahmed - 220042163*
4. *Navid Ibrahim - 220042101*

Institution

*Islamic University of Technology
Department of CSE*

Course

*SWE 4602
Software Design & Architectures
Department of CSE*

Submission Date

July 10, 2026

Contents

1	Description	2
2	Idea	2
3	List of Possible Services	3
4	Justification for Separation into Services	3
5	Chosen Services and Microservice Interaction	4
5.1	Chosen Services	4
5.2	Interaction Overview	4
6	System Architecture Diagram	5
7	Use Case Diagram	6
8	Tech Stack	6
9	Conclusion	7

1 Description

This project presents the design and implementation of a **Ticket Booking System**, inspired by platforms such as Ticketmaster (<https://www.ticketmaster.com/>). The system allows users to browse events, view seating availability, book tickets, make payments, and receive booking confirmations.

The goal of this project is to demonstrate the principles of **microservices architecture** in a realistic, moderately complex domain. Rather than building a single monolithic application, the system is decomposed into independently deployable services, each responsible for a specific business capability. The project also demonstrates key architectural patterns commonly used in production-grade distributed systems, including:

- Service registration and discovery
- API Gateway routing and cross-cutting concerns (authentication, rate limiting)
- Synchronous inter-service communication (REST / OpenFeign)
- Asynchronous, event-driven communication (message broker)
- Independent data ownership per service

2 Idea

The core idea is to simulate an end-to-end ticket booking flow: a user searches for an event, selects seats, and completes a purchase. Behind this seemingly simple user journey lies a set of concerns that map naturally onto separate services – user identity, event/catalog data, seat inventory and locking, order/booking orchestration, payment processing, and notifications.

By separating these concerns into services, the project demonstrates:

- How a single user action (booking a ticket) can trigger a chain of inter-service calls.
- How synchronous calls (e.g., checking seat availability) differ in design from asynchronous ones (e.g., sending a confirmation email).
- How service discovery and an API Gateway simplify client-side complexity and enable horizontal scaling.
- How data consistency challenges (e.g., double-booking a seat) are handled in a distributed system.

3 List of Possible Services

The table below lists all services that were considered during the design phase. Not all of them were implemented in the final system (see Section 5), but they are documented here to show the full scope of a real-world ticket booking platform.

Service	Responsibility
User Service	User registration, authentication, login, profile management, JWT issuing/validation.
Event/Catalog Service	Manages events, venues, artists, categories, and showtimes. Supports browsing and filtering.
Venue / Seat Management Service	Maintains seating layout (sections, rows, seats) for each venue and showtime.
Booking / Order Service	Orchestrates the booking workflow – creates orders, coordinates seat locking and payment, tracks order status.
Inventory / Seat Locking Service	Temporarily locks selected seats during checkout to prevent double-booking; releases locks on timeout/failure.
Payment Service	Processes payments (mock gateway), handles payment success/failure, refunds.
Notification Service	Sends asynchronous email/SMS confirmations and reminders.
Search Service	Full-text / faceted search over events (e.g., Elasticsearch-backed).
Pricing / Discount Service	Dynamic pricing rules, promo codes, discounts.
Review / Rating Service	Allows users to rate and review events after attendance.
Admin / Reporting Service	Analytics dashboard for organizers – sales figures, occupancy rates.
API Gateway	Single entry point for all client requests; routing, authentication filtering, rate limiting.
Service Registry (Eureka)	Enables dynamic service discovery so services can locate each other without hardcoded addresses.
Config Server	Centralized, externalized configuration management for all services.

4 Justification for Separation into Services

Each service was evaluated against the question: *does this responsibility have a distinct data ownership, scaling need, security boundary, or lifecycle from the rest of the system?* The justifications below explain why the more significant services were kept separate rather than merged.

- **User Service:** Authentication and identity management is a cross-cutting concern with strict security requirements. Isolating it creates a clear security boundary and allows independent scaling of login traffic versus business traffic.
- **Event/Catalog Service:** This service is read-heavy and changes on a different cadence (organizers updating event details) than the booking workflow. Isolating it allows aggressive caching and independent scaling for browsing traffic, which is typically much higher than booking traffic.
- **Booking/Order Service:** Represents the core transactional workflow of the system. It requires strong consistency guarantees (avoiding double-booking, ensuring payment before confirmation), which is a very different concern from the read-optimized catalog service.

- **Inventory/Seat Locking Service** (*if implemented separately*): This is a hot, latency-sensitive path prone to race conditions when many users attempt to book the same seat concurrently. Isolating it allows the use of specialized techniques (distributed locks, short-lived reservations, e.g., via Redis) without affecting the rest of the booking logic.
- **Payment Service**: Payment handling has compliance implications (e.g., PCI-DSS in a real system) and typically integrates with external gateways. Isolating it limits the security "blast radius" and keeps sensitive logic auditable and swappable independent of the rest of the system.
- **Notification Service**: Sending emails/SMS is a "fire-and-forget" side effect that should not block the critical booking path. It is naturally suited to asynchronous, event-driven communication and can be scaled or retried independently of booking logic.
- **API Gateway & Service Registry**: These are not business services but architectural necessities. The Gateway centralizes cross-cutting concerns (auth, routing, rate limiting) so individual services do not duplicate this logic. The Service Registry removes the need for hardcoded service addresses, allowing services to scale up/down or move without reconfiguring clients.

5 Chosen Services and Microservice Interaction

5.1 Chosen Services

For the scope of this course project, the following services were selected for implementation:

1. **User Service** – registration, login, JWT-based authentication, profile management.
2. **Event Service** – manages events, venues, showtimes, and seat availability data.
3. **Booking Service** – orchestrates the booking workflow: validates seat availability, creates orders, coordinates payment, confirms/cancels bookings.
4. **Payment Service** – simulates payment processing and emits payment result events.
5. **Notification Service** – consumes booking/payment events asynchronously and sends confirmation emails.

Supporting infrastructure components:

- **API Gateway** (Spring Cloud Gateway) – single entry point for the frontend.
- **Service Registry** (Netflix Eureka) – service discovery for all microservices.
- **Config Server** (Spring Cloud Config) – *optional* centralized configuration.
- **Message Broker** (RabbitMQ / Kafka) – for asynchronous events between Booking, Payment, and Notification services.

5.2 Interaction Overview

Synchronous communication (REST, via OpenFeign, routed through the Gateway):

- Frontend → API Gateway → routed to the target service based on path/JWT claims.
- Booking Service → Event Service: checks seat availability and retrieves pricing information.
- Booking Service → User Service: validates the authenticated user (or JWT is validated directly at the Gateway).
- Booking Service → Payment Service: submits a payment request synchronously and awaits an immediate accept/reject response.

Asynchronous communication (event-driven, via message broker):

- Payment Service publishes a `PaymentCompleted` or `PaymentFailed` event → consumed by Booking Service to finalize or cancel the order.
- Booking Service publishes a `BookingConfirmed` event → consumed by Notification Service, which sends a confirmation email.

Typical end-to-end booking flow:

1. User browses events: Frontend → Gateway → Event Service.
2. User selects seats and initiates checkout: Frontend → Gateway → Booking Service.
3. Booking Service calls Event Service to re-verify seat availability and lock the seat.
4. Booking Service creates a `PENDING` order and calls Payment Service synchronously.
5. Payment Service processes payment and returns a result; it also emits an async event.
6. On success, Booking Service marks the order `CONFIRMED` and emits a `BookingConfirmed` event.
7. Notification Service consumes the event and sends a confirmation email to the user.

6 System Architecture Diagram

Figure 1 illustrates the overall system architecture, showing the frontend, API Gateway, Service Registry, the five core microservices, their databases, and the message broker used for asynchronous communication.

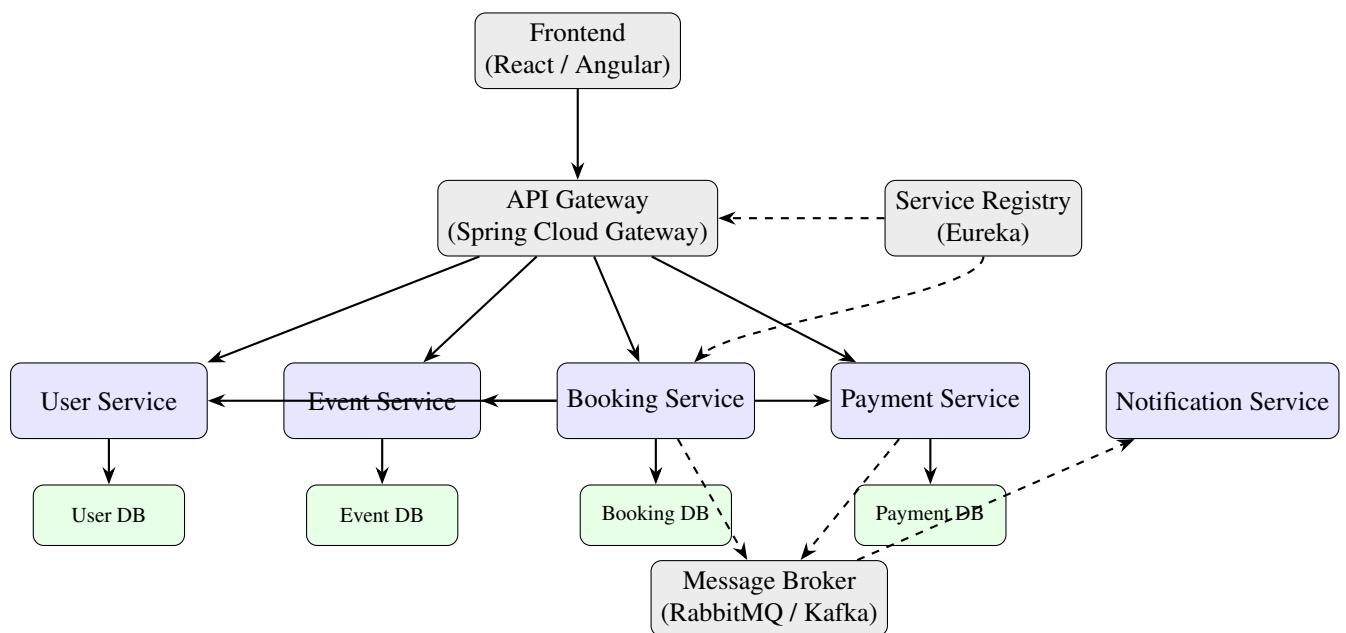


Figure 1: System architecture: solid arrows represent synchronous REST calls; dashed arrows represent asynchronous/event-driven communication or service discovery.

7 Use Case Diagram

Figure 2 shows the primary use cases for the chosen services, covering the main actor (Registered User) and the interactions with the system.

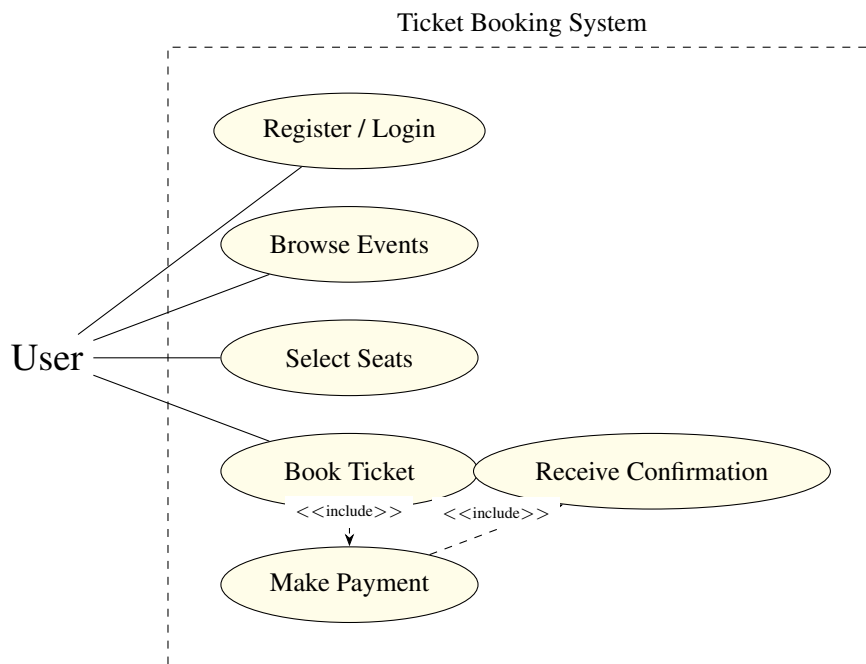


Figure 2: Use case diagram for the Ticket Booking System covering the chosen services.

8 Tech Stack

Layer	Technology
Backend Framework	Spring Boot (Java)
Service Discovery	Netflix Eureka (Spring Cloud Netflix)
API Gateway	Spring Cloud Gateway
Configuration	Spring Cloud Config Server (<i>optional</i>)
Inter-service Sync Communication	REST APIs via OpenFeign / RestTemplate/ WebClient
Inter-service Async Communication	RabbitMQ or Apache Kafka
Database	PostgreSQL / MySQL (one database per service, following the Database-per-Service pattern)
Caching (<i>optional</i>)	Redis (e.g., for seat-lock reservations)
Authentication	JWT (JSON Web Tokens), Spring Security
Frontend	React.js (or Angular), Axios for API calls
Containerization	Docker, Docker Compose
API Documentation	Swagger / OpenAPI
Build Tool	Maven / Gradle
Version Control	Git, GitHub

9 Conclusion

This report outlined the design of a Ticket Booking System built on a microservices architecture. By decomposing the system into services with clear responsibilities – User, Event, Booking, Payment, and Notification – and supporting them with a Service Registry, API Gateway, and both synchronous and asynchronous communication mechanisms, the project demonstrates the core principles taught in the Software Design and Architectures course while remaining scoped appropriately for a semester project.